Part PT

Project Task

PT.1 Project Task Formalities

In this part, you will find the project task for the course. The project is a significant component of the course examination (30 of 100 Points) and allows you to apply the concepts and methods learned throughout the lectures and exercises to a real-world problem. There are a few ground rules:

- You are allowed to work in groups of two students (but not required to do so).
- However, you must submit your project solution individually (via isis).
  - You have to indicate in your submission that you worked together with another student
  - t the top of the run file, you should change the dummy numbers to the actual matriculation numbers of the group members (see the file in the project folder).
  - If you do not submit a project solution, you will receive 0 points for the project part of the examination (even if your group member submits a solution).
- Deadline for submission is end of March. Late submissions will not be accepted.

The project is handed out as a template file. This file consists of the following components:

- `Data`: folder containing the relevant data for the project.
- `src.jl`: Julia file containing the skeleton code for the project. It includes a few functions already, that show you in an exemplary fashion how to implement the required functions.
- `test.jl`: Julia file containing a test script. This allows you to check whether your implementation is correct. You can run this file after you have implemented the required functions in src.jl. If all tests pass, you can be confident that your implementation is correct (unless you hard-coded the tests, obviously).

- `run.jl`: Julia file containing the main script for the project. You can run this file to execute your project solution.
- `test_data`: folder containing loadable Julia data structures that contain the target solutions. You can use these data structures to check your results against the expected outcomes.
  - ll data structures in the `test_data` folder are saved with the `JLD2.jl` package. You can load them with the following code:
    `@load joinpath(@__DIR__, "test\_data", "data.jld2") data`
  - For further information on how to use the `JLD2.jl` package, see the documentation at https://juliaio.github.io/JLD2.jl/stable/.
- `results`: folder where you are required to save your results (structured the same way as the `test_data` folder).
- `manifest.toml` and a `project.toml` file for the Julia environment. You can use this environment to run the project code.
  - *Warning: You are not allowed to use any other packages than those specified in the project environment (except for a different solver, if you do not want to obtain an academic license for Gurobi).*

Upon submission, it is required that you rename the template folder to your matriculation number (six digits only)!

s introduced in the last lecture session, you will work on the task of dimensioning a 100 % renewable power system for a simplified model of Germany. The goal is to determine the optimal capacities of generation and storage technologies as well as the optimal operation of these technologies over the course of one year at hourly resolution for various market configurations and weather years. You will work with real data provided by the federal network agency (Bundesnetzagentur) for each of the four German electricity Transmission System Operators (TSOs).[1] The following tasks will take you through the necessary steps to implement some suitable models in Julia.

[1] See https://www.smard.de/home/downloadcenter/download-marktdaten/ to downlaod the data yourself, and https://www.smard.de/page/en/wiki-article/5884/205528/transmission-system-operators for more graphical detail on the TSO areas.

PT.2 Project Task Descriptions

PT.2.1 Data Preparation (lready Implemented in src.jl)

This part of the project is already implemented in the provided `sr .jl` *file. You can use this as a reference for how to implement the required functions.*

Write a function `combine_data()` that reads in the provided csv files inside the `Data` folder and combines them into a single dataframe (of your choice) that holds all relevant information for further processing. You may use the provided test data structure `test_data/data.jld2` to check whether your implementation is correct. You should proceed along the following lines:

- Read in generation, demand, capacity and price data from the respective csv files for all four German TSO areas.
  - There are actually a few caveats in this, including (but not limited to) how to work with summer/winter time etc. and renaming some columns
  - You should replace all missing values by zeros.
- Join the data into a single DataFrame that holds the following columns:
  - `Start date`
  - `Year`
  - `Month`
  - `Day`
  - `hour_of_year`
  - `  rea`
  - `grid load [MWh]`
  - `Wind offshore [MWh]`
  - `Wind onshore [MWh]`

  - `Photovoltaics [MWh]`
  - `Germany/Luxembourg [€/MWh]`
  - `Wind offshore [MW]`
  - `Wind onshore [MW]`
  - `Photovoltaics [MW]`
- Use the available data to calculate the availability of Wind offshore, Wind onshore, and Photovoltaics. The availability is the ratio of generation to installed capacity.
  - Be careful, not all data is provided at the same granularity. To get valid capacities, you may assume that all new generation is mysteriously installed between the last hour of any given year and the first hour of the following one.[1]
- Then, similar to the exercise in Section E.8.2, you should use the price and load data to calculate the parameters of a first order approximation to the inverse demand function for each TSO area. You may use a price elasticity of demand of $\sigma = \quad 0.2$ for all areas.

$$p\left(Q\right) = a + b * \left(Q\right) \tag{PT.2.1a}$$

$$b = \frac{p^{ref}}{Q^{ref}} \cdot \frac{1}{\sigma} \tag{PT.2.1b}$$

$$a = p^{ref} \quad b \cdot Q^{ref} = p^{ref} \quad \frac{p^{ref}}{\sigma} \tag{PT.2.1c}$$

- Finally, save the resulting DataFrame with `JLD2.jl` as `joinpath(@__DIR__, "results", "data.jld2")`.

Whenever you are not exactly sure about what is supposed to come out of the function, you can always check the target data sheet `test_data/data.jld2` to see what the expected output looks like.

[1] This will later on lead to a potential availability of more than 1. However, this is not an issue for our project.

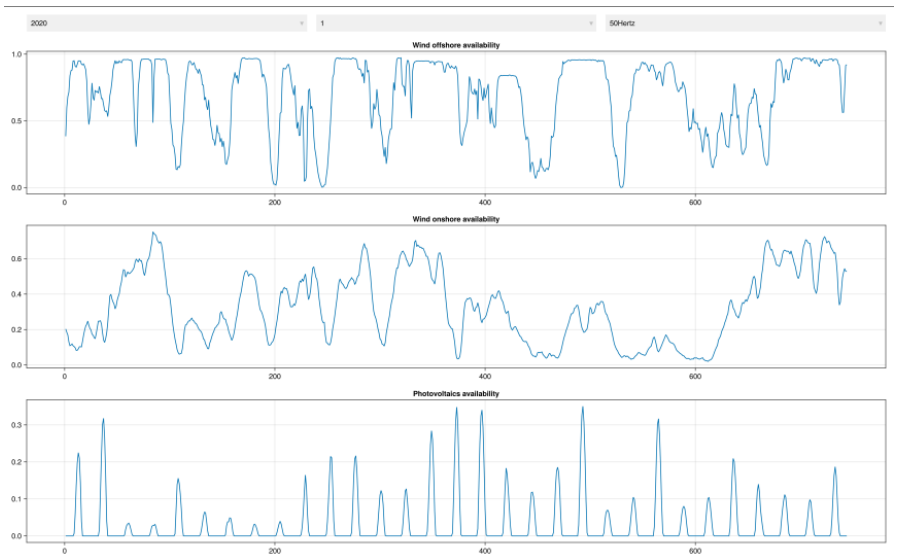

Figure PT.2.1: Example of a data inspection visualization for the availability data.

PT.2.2 Data Preparation visualization (lready Implemented in src.jl)

This part of the project is already implemented in the provided `sr .jl` *file. You can use this as a reference for how to implement the required functions.*

Write a function `create_data_inspection()` that creates a visualization that allows you to inspect the availability data for each TSO area, weather year and month. It should look pretty similar to Figure PT.2.1. Your function should not take an argument, but should read in the data from `joinpath(@__DIR__, "results", "data.jld2")`. If you did not manage to create the data file, you can use the provided test data structure `test_data/data.jld2` to write this.

PT.2.3 Cost Optimization Model (7.5 Points)

Implement a linear optimization model for a multi-zone power system with endogenous investment and operational decisions. Define the following index sets:

- $\mathcal{G}s$: generation technologies (Wind offshore, Wind onshore, Photovoltaics)
- $\mathcal{S}s$: storage technologies (Batteries, Hydrogen)
- $\mathcal{Z}s$: zones (four German TSO areas)
- $\mathcal{T}s$: time steps (8760 hours in a year)
- and a mapping that assigns to each hour the previous one (as during the lecture the hour before the first hour is the last one).

Then introduce the following non-negative continuous variables:

- Investment variables (capacity decisions):
  - Installed generation capacity per technology and zone
  - Installed storage energy capacity per storage technology and zone
  - Installed injection (charging) power capacity per storage and zone
  - Installed extraction (discharging) power capacity per storage and zone
- Operational variables (time-dependent):
  - Generation output per technology, zone, and time
  - Injection (charging) per storage, zone, and time
  - Extraction (discharging) per storage, zone, and time
  - Storage state of charge per storage, zone, and time
  - Power flow between every pair of zones and time, bounded above by an exogenous transmission capacity parameter

The objective is to minimize total system cost consisting of:

- Generation investment costs (capacity times technology-specific unit cost)

- Storage energy capacity investment costs
- Injection power capacity investment costs
- Extraction power capacity investment costs
- Variable generation costs (dispatch times marginal cost)
- Linear transmission flow costs (flow times per-unit flow cost)

nd is constrained by:

- Generation vailability Constraint: For every generation technology, zone, and time generation output must not exceed installed generation capacity multiplied by an exogenous availability factor (as calculated in your data).
- Injection Power Limit: For every storage technology, zone, and time charging power must not exceed installed injection capacity.
- Extraction Power Limit: For every storage technology, zone, and time: Discharging power must not exceed installed extraction capacity.
- Storage Energy Capacity Limit: For every storage technology, zone, and time state of charge must not exceed installed storage energy capacity.
- Storage Dynamics: For every storage technology, zone, and time state of charge at time t equals state of charge at previous time plus injection at time t minus extraction at time t. ll efficiencies should be handled in the market clearing constriant.
- Market Clearing (Zonal Energy Balance): For every zone and time total supply must be greater than or equal to total demand.
  - Left-hand side (supply):
    - Sum of generation in the zone
    - Sum of storage extraction multiplied by discharge efficiency
    - Sum of inflows from all other zones
  - Right-hand side (demand):
    - Exogenous demand
    - Injection divided by charging efficiency
    - Sum of outflows to all other zones

Table PT.2.1 provides an overview of the sets required.

Set	Elements
Storage technologies S	Battery, Hydrogen
Generation technologies G	Wind offshore, Wind onshore, Photovoltaics, Loss of Load
Zones Z	50Hertz, mprion, TenneT, TransnetBW
Time steps T	$t = 1, \ldots, 8760$

Table PT.2.1: Model Sets for Project Task.

Table PT.2.2 depicts the marginal generation and investment costs for the different generation technologies. The cost of loss of load is set to 1000 /MWh and modeled as a generation technology. However, since we are only modeling a single year, we have to annuitize the investment costs for generation and storage technologies. You can use a discount rate of 5% and assume a lifetime of 30 years on all assets, except for batteries where you can assume a lifetime of 15 years. The annuity factor can be calculated as follows:

$$\text{annuity_factor}(r, n) = \frac{r}{1 \quad (1 + r)^{(} \quad n)} \tag{PT.2.2}$$

Technology	Investment cost [€/MW]	Marginal cost [€/MWh]
Wind offshore	$2.8 \cdot 10^6$	1
Wind onshore	$1.2 \cdot 10^6$	1
Photovoltaics	$0.6 \cdot 10^6$	1
Loss of Load	1	1000

Table PT.2.2: Generation investment and marginal costs

Table PT.2.3 depicts the investment costs for storage energy capacity, injection power capacity, and extraction power capacity for the two storage technologies. Further, charging and discharging efficiencies are indicated. The marginal costs of storage operation are set to zero.

Transport capacities are scenario dependent. In one case, you must assume, that no transport between the zones is possible. In the other case, you can assume that transport capacities are sufficiently large such that they do not bind (e.g., 100 GW between every

Technology	Capacity [€/MWh]	Injection [€/MW]	Extraction [€/MW]	Inj eff	Ext eff
Battery	$0.3 \cdot 10^6$	1	1	0.95	0.95
Hydrogen	$0.003 \cdot 10^6$	$1.4 \cdot 10^6$	$0.6 \cdot 10^6$	0.60	0.60

Table PT.2.3: Storage investment costs and efficiencies

pair of zones). Now make use of the calculated availabilities and demand parameters, to solve the optimization problem for both transport scenarios and all weather years (2020-2025) individually. You should then solve the outcomes of all combinations of transport scenarios and weather years into a dict. This dict should :

- Use the weather year as the first key and the transport scenario as the second key (e.g., `results[2020][true]` should give you the results for the 2020 weather year and local pricing (i.e. no transmission capacity) scenario)
- Hold a dict with the following information for each scenario combination:
  - "extraction": result for the extraction variable
  - "demand": result for the demand (exogenous data), but as a Dense xis rray with the same structure as the other variables (i.e., indexed by zone and time)
  - "storage_level": result for the storage level variable
  - "injection_capacity": result for the injection capacity variable
  - "prices": result for the prices (i.e., duals to the market clearing constraint)
  - "injection": result for the injection variable
  - "generation": result for the generation variable
  - "storage_capacity": result for the storage capacity variable
  - "generation_capacity": result for the generation capacity variable
  - "total_cost": the optimal objective value of the cost minimization problem
  - "flow": result for the flow variable
  - "extraction_capacity": result for the extraction capacity variable
  - You can use the provided test data structure `test_data/deterministic_cost_minimization_results.jld2` to check whether your implementation is correct.[2]

[2] Be careful of LP degeneracy here. Simply check for the optimal objective value and the general structure of the solution (e.g., which technologies are used etc.) rather than checking for exact values of all variables.

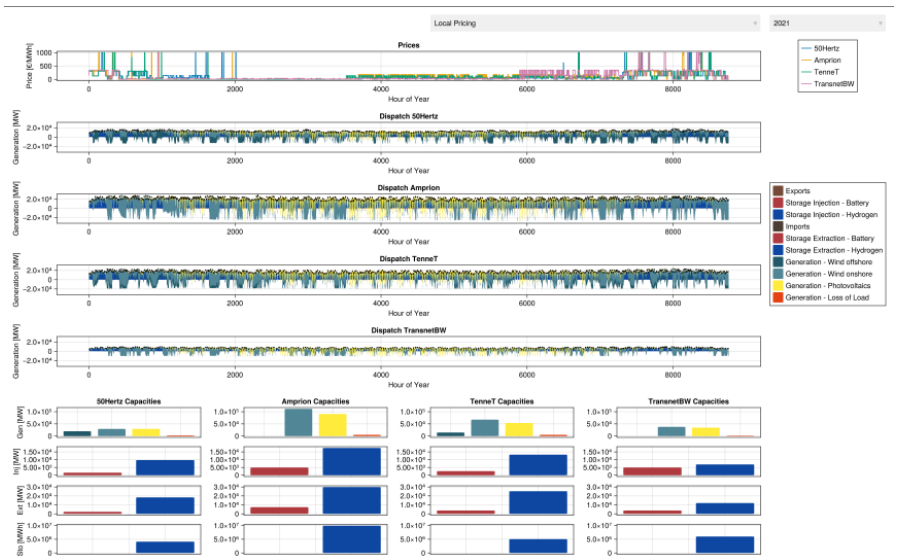

Figure PT.2.2: Example of a cost optimization visualization for the deterministic model.

Your function `get_deterministic_cost_minimization_results(optimizer)` should save a dict exactly like `test_data/deterministic_cost_minimization_results.jld2` and return nothing.

PT.2.4 Visualization of Cost Optimal Solution (2.5 Points)

Write a function `create_deterministic_cost_minimization_results_visualization()` that creates a visualization that allows you to inspect the dispatch and capacity investments. It should look pretty similar to Figure PT.2.2. Your function can either take the grid constraint and weather year as arguments, or include them in an interactive plot using `GLMakie`.[3] However, it should read in the data from `joinpath(@__DIR__, "results")`. If you did not manage to create the data file (or decided to do the plotting first), you

[3]Similar to the function `create_data_inspection()` already implemented in `src.jl`

can use the provided test data structure in `test_data` to write this function. The plotting colors are given by Listing PT.2.1.

Listing PT.2.1: Term Project Color Map

```
cmap = Dict(
        "Storage Injection - Battery" => "#ae393f",
        "Storage Injection - Hydrogen" => "#0d47a1",
        "Storage Extraction - Battery" => "#ae393f",
        "Storage Extraction - Hydrogen" => "#0d47a1",
        "Imports" => "#4d3e35",
        "Exports" => "#754937",
        "Generation - Loss of Load" => "#e54213",
        "Generation - Wind offshore" => "#215968",
        "Generation - Wind onshore" => "#518696",
        "Generation - Photovoltaics" => "#ffeb3b",
    )
```

PT.2.5 Welfare Optimization Model (7.5 Points)

Now turn your model into a welfare optimization model. Write a function `get_deterministic_welfare_maximization_results(optimizer)` that implements the necessary changes to the cost optimization model to turn it into a welfare optimization model. You should now assume, that each TSO operates the assets in its zone.[4] Therefore, the model should now include the following variables:

- Generation capacity of each firm for each generation technology located in each zone (i.e. 3 axes: firm, technology, location)
- Storage capacity of each storage technology in each zone (i.e. 2 axes: storage technology, location)

[4] This is an obvious simplification and does not at all capture how the real electricity market works.

- Injection capacity of each storage technology in each zone (i.e. 2 axes: storage technology, location)
- Extraction capacity of each storage technology in each zone (i.e. 2 axes: storage technology, location)
- Storage level of each firm for each storage technology in each zone and time period (i.e. 4 axes: firm, storage technology, location, time)
- Generation of each firm for each generation technology in each zone and time period (i.e. 4 axes: firm, technology, location, time)
- Injection of each firm for each storage technology in each zone and time period (i.e. 4 axes: firm, storage technology, location, time)
- Extraction of each firm for each storage technology in each zone and time period (i.e. 4 axes: firm, storage technology, location, time)
- Flow of each firm from origin zone to destination zone in each time period (i.e. 4 axes: firm, origin, destination, time)
- Sales Q of each firm in each zone and time period (i.e. 3 axes: firm, location, time)

Then, solve the model for the same scenarios as before and save the results into a dict called `results/deterministic_welfare_maximization_results.jld2`. It should look like `test_data/deterministic_welfare_maximization_results.jld2`. The dict should have the following structure:

- Use the weather year as the first key and the transport scenario as the second key (e.g., `results[2020][true]` should give you the results for the 2020 weather year and local pricing (i.e. no transmission capacity) scenario)
- Hold a dict with the following information for each scenario combination:
  - "extraction": result for the extraction variable
  - "demand": result for the demand (endogenously determined), as a Dense x-is rray with the same structure as the other variables (i.e., indexed by zone and time)
  - "storage_level": result for the storage level variable
  - "injection_capacity": result for the injection capacity variable

- "prices": result for the prices (calculated from the inverse demand function using the sales variable)
- "injection": result for the injection variable
- "generation": result for the generation variable
- "storage_capacity": result for the storage capacity variable
- "generation_capacity": result for the generation capacity variable
- "total_cost": the optimal objective value of the maximization problem[5]
- "flow": result for the flow variable
- "extraction_capacity": result for the extraction capacity variable
- The dict should be structurally similar to the linear cost minimization case, but obviously some of the variables would have more axes.

PT.2.6 Strategic Behavior (7.5 Points)

Now, derive an adequate convex reformulation of the equilibrium/complementarity problem when each zone is represented by a strategic player that maximizes its own profit. You should use the TSO's as in the welfare optimization task, but this time with market power. Implement the reformulation in a function `get_deterministic_strategic_behavior_results(optimizer)` and solve the model for the same scenarios as before. Save the results into a dict called `results/deterministic_s` It should look like `test_data/deterministic_strategic_behavior_results.jld2`, and be structured like the dict of the welfare optimization task.

PT.2.7 Visualization of Elastic Models (5 Points)

Write a function `create_deterministic_elastic_results_visualization()` that creates a

[5] Note that this is not cost in the economic sense.

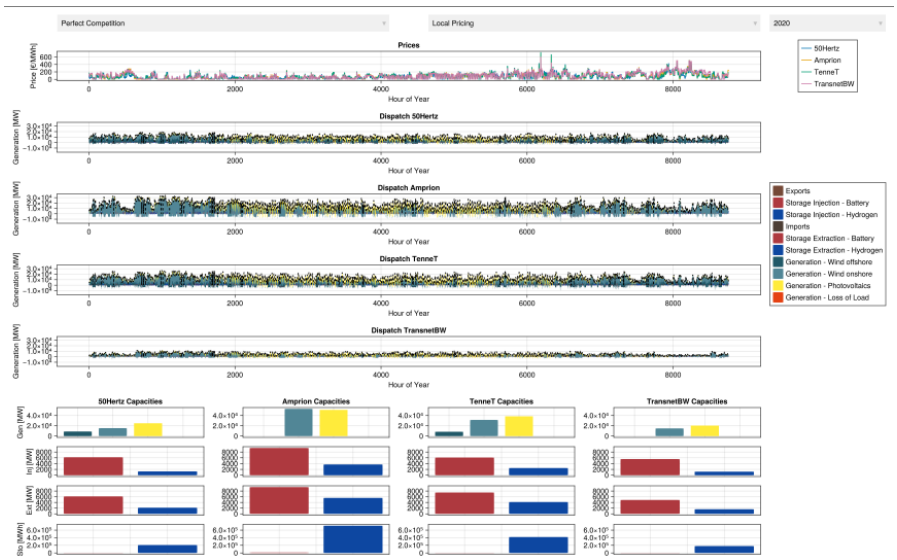

Figure PT.2.3: Example of a welfare optimization visualization for the deterministic model.

visualization that allows you to inspect the dispatch and capacity investments. It should look pretty similar to Figure PT.2.3. Your function can either take the grid constraint, behavior, and weather year as arguments, or include them in an interactive plot using `GLMakie`. However, it should read in the data from `joinpath(@__DIR__, "results")`. If you did not manage to create the data file, you can use the provided test data structure in `test_data` to write this function.

PT.2.8 Bonus: Stochastic Dual Dynamic Programming (5 Points)

Create a two stage stochastic optimization problem and implement a solution with `SDDP.jl` in the function `get_stochastic_cost_minimization_results(optimizer, local_pricing, iteration_limit = 1, lower_bound = -Inf,)`. Your model

should have investment decisions on the first stage, and operational decisions on the second stage. Uncertainty should be represented by a set of scenarios that differ in their availability of renewable generation and demand. You can use the provided test data structure `test_data/stochastic_cost_minimization_results.jld2` to check whether your implementation yields the right structures.[6] Finally, also visualize your results in a similar way to Figure PT.2.3, using a function `create_stochastic_cost_minimization_results_visualization()`.

[6] Obviously, the model is not solved to optimality within 1000 iterations, so there is little sense in comparing objective values

PT.3 Presentation of Template Structure

The `run.jl` script is depicted in Listing PT.3.1. In addition, Listing PT.3.2 depicts the test script `test.jl`. You can use this to check whether your implementation is correct. If all tests pass, you can be confident that your implementation is correct (unless you hard-coded the tests, obviously).

Listing PT.3.1: Term Project Run Script

```
# 123456, 098765 (mat no of group members)

include(joinpath(@__DIR__, "src.jl"))

optimizer = Gurobi.Optimizer

combine_data()
create_data_inspection()

get_deterministic_cost_minimization_results(optimizer)
create_deterministic_cost_minimization_results_visualization()

get_deterministic_welfare_maximization_results(optimizer)
get_deterministic_strategic_behavior_results(optimizer)
create_deterministic_elastic_results_visualization()

get_stochastic_cost_minimization_results(optimizer, true, 100, 1e+10)
create_stochastic_cost_minimization_results_visualization()
```

Listing PT.3.2: Term Project Test Script

```
using JLD2
using Test
using DataFrames
using JuMP
using SDDP

```

```
function test_data_loading()

    test_data = load(joinpath(@__DIR__, "test_data", "data.jld2"), "data")
    result_data = load(joinpath(@__DIR__, "results", "data.jld2"), "data")

    joincols = ["Start date", "Year", "Month", "Day", "hour_of_year", " rea"]
    @test joincols ⊆ names(result_data)
    jdf = leftjoin(result_data, test_data, on = joincols, makeunique = true)

    for col in setdiff(names(test_data), joincols)
        @test jdf[!, "$(col)_1"] ≈ jdf[!, "$(col)"] rtol = 1e-3
        @test !(any(ismissing(result_data[!, col])))
    end

end

function test_deterministic_cost_minimization_results()

    test_data = load(
        joinpath(@__DIR__, "test_data", "deterministic_cost_minimization_results
            .jld2"),
        "deterministic_cost_minimization_results",
    )
    result_data = load(
        joinpath(@__DIR__, "results", "deterministic_cost_minimization_results.
            jld2"),
        "deterministic_cost_minimization_results",
    )

    for y = 2020:2025, local_pricing in [true, false]
        @test is (
            result_data[y, local_pricing]["extraction"],
            JuMP.Containers.Dense xis rray{Float64,3},
        )
        @test is (
            result_data[y, local_pricing]["demand"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
            result_data[y, local_pricing]["storage_level"],
            JuMP.Containers.Dense xis rray{Float64,3},
        )
        @test is (
            result_data[y, local_pricing]["injection_capacity"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
```

```
        @test is (
            result_data[y, local_pricing]["prices"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
            result_data[y, local_pricing]["injection"],
            JuMP.Containers.Dense xis rray{Float64,3},
        )
        @test is (
            result_data[y, local_pricing]["generation"],
            JuMP.Containers.Dense xis rray{Float64,3},
        )
        @test is (
            result_data[y, local_pricing]["storage_capacity"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
            result_data[y, local_pricing]["generation_capacity"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
            result_data[y, local_pricing]["flow"],
            JuMP.Containers.Dense xis rray{Float64,3},
        )
        @test is (
            result_data[y, local_pricing]["extraction_capacity"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test isapprox(
            result_data[y, local_pricing]["total_cost"],
            test_data[y, local_pricing]["total_cost"],
            rtol = 1e-3,
        )
    end
end

function test_deterministic_welfare_maximization_results()

    test_data = load(
        joinpath(@__DIR__, "test_data", "
            deterministic_welfare_maximization_results.jld2"),
        "deterministic_welfare_maximization_results",
    )
    result_data = load(
        joinpath(@__DIR__, "results", "
            deterministic_welfare_maximization_results.jld2"),
```

```
        "deterministic_welfare_maximization_results",
    )

    for y = 2020:2025, local_pricing in [true, false]
        @test is (
            result_data[y, local_pricing]["extraction"],
            JuMP.Containers.Dense xis rray{Float64,4},
        )
        @test is (
            result_data[y, local_pricing]["demand"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
            result_data[y, local_pricing]["storage_level"],
            JuMP.Containers.Dense xis rray{Float64,4},
        )
        @test is (
            result_data[y, local_pricing]["injection_capacity"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
            result_data[y, local_pricing]["prices"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
            result_data[y, local_pricing]["injection"],
            JuMP.Containers.Dense xis rray{Float64,4},
        )
        @test is (
            result_data[y, local_pricing]["generation"],
            JuMP.Containers.Dense xis rray{Float64,4},
        )
        @test is (
            result_data[y, local_pricing]["storage_capacity"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
            result_data[y, local_pricing]["generation_capacity"],
            JuMP.Containers.Dense xis rray{Float64,3},
        )
        @test is (
            result_data[y, local_pricing]["flow"],
            JuMP.Containers.Dense xis rray{Float64,4},
        )
        @test is (
            result_data[y, local_pricing]["extraction_capacity"],
```

```
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test isapprox(
            result_data[y, local_pricing]["total_cost"],
            test_data[y, local_pricing]["total_cost"],
            rtol = 1e-3,
        )
    end

end

function test_deterministic_strategic_behavior()

    test_data = load(
        joinpath(@__DIR__, "test_data", "
            deterministic_strategic_behavior_results.jld2"),
        "deterministic_strategic_behavior_results",
    )
    result_data = load(
        joinpath(@__DIR__, "results", "deterministic_strategic_behavior_results.
            jld2"),
        "deterministic_strategic_behavior_results",
    )

    for y = 2020:2025, local_pricing in [true, false]
        @test is (
            result_data[y, local_pricing]["extraction"],
            JuMP.Containers.Dense xis rray{Float64,4},
        )
        @test is (
            result_data[y, local_pricing]["demand"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
            result_data[y, local_pricing]["storage_level"],
            JuMP.Containers.Dense xis rray{Float64,4},
        )
        @test is (
            result_data[y, local_pricing]["injection_capacity"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
            result_data[y, local_pricing]["prices"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
```

```
            result_data[y, local_pricing]["injection"],
            JuMP.Containers.Dense xis rray{Float64,4},
        )
        @test is (
            result_data[y, local_pricing]["generation"],
            JuMP.Containers.Dense xis rray{Float64,4},
        )
        @test is (
            result_data[y, local_pricing]["storage_capacity"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test is (
            result_data[y, local_pricing]["generation_capacity"],
            JuMP.Containers.Dense xis rray{Float64,3},
        )
        @test is (
            result_data[y, local_pricing]["flow"],
            JuMP.Containers.Dense xis rray{Float64,4},
        )
        @test is (
            result_data[y, local_pricing]["extraction_capacity"],
            JuMP.Containers.Dense xis rray{Float64,2},
        )
        @test isapprox(
            result_data[y, local_pricing]["total_cost"],
            test_data[y, local_pricing]["total_cost"],
            rtol = 1e-3,
        )
    end
end

function test_stochastic_cost_minimization_results()

    result_data = load(
        joinpath(@__DIR__, "results", "stochastic_cost_minimization_results.jld2
            "),
        "stochastic_cost_minimization_results",
    )

    for (idx, y) in enumerate(2020:2025)
        @test result_data[idx][2][:noise_term] == y
        @test is (
            result_data[idx][2][:injection],
            JuMP.Containers.Dense xis rray{Float64,3},
        )
        @test is (
```

```
            result_data[idx][2][:extraction],
            JuMP.Containers.Dense xis rray{Float64,3},
        )
        @test is (
            result_data[idx][2][:storage_capacity],
            JuMP.Containers.Dense xis rray{SDDP.State{Float64},2},
        )
        @test is (
            result_data[idx][2][:generation],
            JuMP.Containers.Dense xis rray{Float64,3},
        )
        @test is (result_data[idx][2][:flow], JuMP.Containers.Dense xis rray{
            Float64,3})
        @test is (result_data[idx][2][:prices], JuMP.Containers.Dense xis rray{
            Float64,2})
        @test is (
            result_data[idx][2][:extraction_capacity],
            JuMP.Containers.Dense xis rray{SDDP.State{Float64},2},
        )
        @test is (
            result_data[idx][2][:generation_capacity],
            JuMP.Containers.Dense xis rray{SDDP.State{Float64},2},
        )
        @test is (result_data[idx][2][:demand], JuMP.Containers.Dense xis rray{
            Float64,2})
        @test is (
            result_data[idx][2][:injection_capacity],
            JuMP.Containers.Dense xis rray{SDDP.State{Float64},2},
        )

    end
end
@testset " ll Tests" begin
    @testset "Data Loading" begin
        test_data_loading()
    end
    @testset "Deterministic Cost Minimization" begin
        test_deterministic_cost_minimization_results()
    end
    @testset "Deterministic Welfare Maximization" begin
        test_deterministic_welfare_maximization_results()
    end
    @testset "Deterministic Strategic Behavior" begin
        test_deterministic_strategic_behavior()
    end
    @testset "Stochastic Cost Minimization" begin
```

```
        test_stochastic_cost_minimization_results()
    end

end
```

Part PS

Project Solution

Part R

References

R.1 References

Birge, John R., and François Louveaux. 2011. *Introduction to Stochastic Programming.* Springer New York. doi: 10.1007/978-1-4614-0237-4. http://dx.doi.org/10.1007/978-1-4614-0237-4.

Boyd, Stephen, and Lieven Vandenberghe. 2004. *Convex Optimization.* Cambridge, UK: Cambridge University Press. https://web.stanford.edu/~boyd/cvxbook/.

Conejo, ntonio J., Enrique Castillo, Roberto Mínguez, and Raquel García-Bertrand. 2006. *Decomposition Techniques in Mathematical Programming: Engineering and Science pplications.* Springer Berlin Heidelberg. doi: 10.1007/3-540-27686-6. https://doi.org/10.1007/3-540-27686-6.

Cormen, Thomas H, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. 2009. *Introduction to lgorithms.* 3rd ed. The MIT Press. London, England: MIT Press.

Dantzig, George B. 1963. *Linear Programming and Extensions.* Princeton, NJ: Princeton University Press.

Dowson, Oscar, and Lea Kapelevich. 2021. "SDDP.jl: Julia Package for Stochastic Dual Dynamic Programming." *INFORMS Journal on Computing* 33 (1): 27–33. doi: 10.1287/ijoc.2020.0987. http://dx.doi.org/10.1287/ijoc.2020.0987.

Mas-Colell, ndreu, Michael Whinston, and Jerry Green. 1995. *Microeconomic Theory.* Oxford University Press.

Meran, Georg, Markus Siehlow, and Christian von Hirschhausen. 2021. *The Economics of Water: Rules and Institutions.* Springer International Publishing. doi: 10.1007/978-3-030-48485-9.

Nocedal, Jorge, and Stephen Wright. 2006. *Numerical Optimization.* 2nd ed. Springer Series in Operations Research and Financial Engineering. New York, NY: Springer.

Siddiqui, S., and S. Gabriel. 2013. “ n SOS1-Based pproach for Solving MPECs with a Natural Gas Market pplication.” *Networks and Spatial Economics* 13 (2): 205–227. doi: 10.1007/s11067-012-9178-y. https://ideas.repec.org/a/kap/netspa/v13y2013i2p205-227.html.